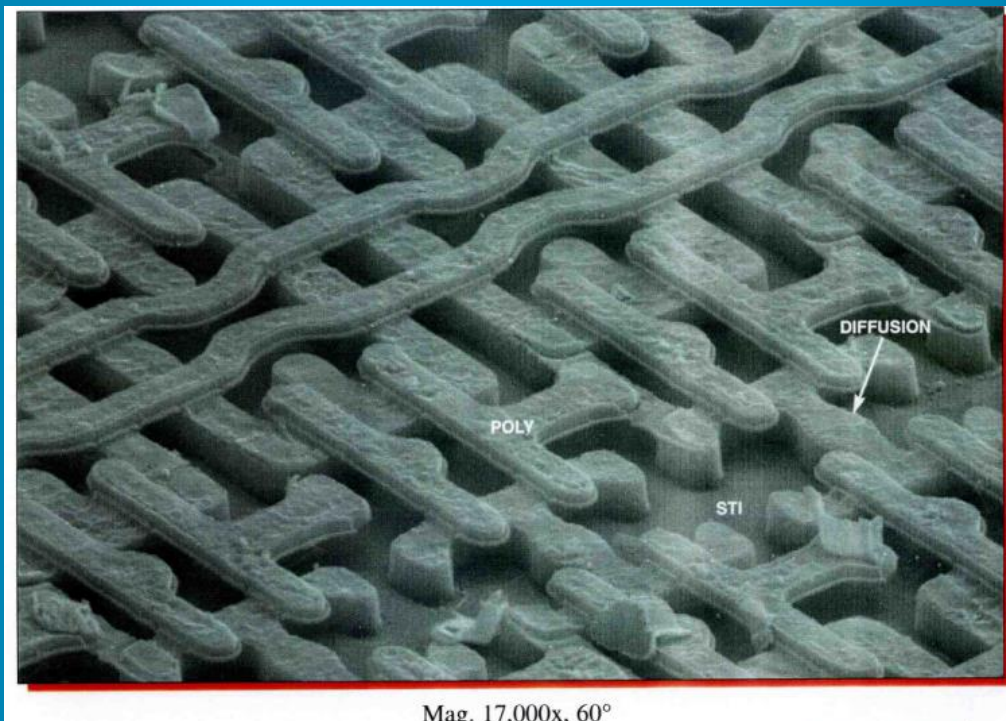


ANALYSER-MODELISER

Systemes à événements discrets (S.E.D.)



Circuit silicium de la société ST - Crolles-Grenoble

Ch 1. Codage de l'information
Ch 2. Modélisation des systèmes discrets

SOMMAIRE :

Chap. I : CODAGE DE L'INFORMATION.....	1
1. Présentation des systèmes numériques de contrôle-commande SNCC	2
2. Systèmes de numération	4
3. Quelques codes élémentaires.....	5
4. Notion de conversion analogique-numérique	7
5. Base de l'algèbre logique	8
 Chap. II : MODELISATION DES SYSTEMES A EVENEMENTS DISCRETS (SED).....	10
1. Introduction aux S.E.D.....	10
2. Chronogramme	10
3. Structure algorithmique	11
4. Modèle SysML pour les SED	11

Ch. 1 : Codage de l'information

1. SYSTEMES NUMERIQUES DE CONTROLE-COMMANDE (SNCC) :

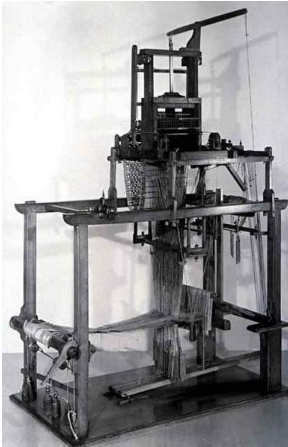
1.1 Un peu d'histoire :

En 1745, Jacques de Vaucanson conçoit le premier métier à tisser entièrement automatique. En 1793, le mécanicien français Joseph-Marie Jacquard s'inspire de ce prototype pour fabriquer un métier à tisser dont les séquences d'opérations à effectuer sont inscrites sur des cartes perforées : l'automatisation appliquée à l'industrie est née. Elle s'épanouira et se généralisera à l'ensemble des activités industrielles dans la première moitié du XXe siècle, en association avec l'instauration de nouvelles méthodes d'organisation scientifique du travail, inspirées des travaux de Taylor. Depuis lors, elle ne cessera de se perfectionner grâce à l'utilisation des techniques issues de l'électronique, de la robotique et de l'informatique.



Jean-Marie JACQUARD,
1752-1834

Aujourd'hui, le développement s'oriente vers des systèmes numériques dits de contrôle-commande (SNCC) que nous détaillerons un peu plus loin. Ces derniers, le plus souvent embarqués, doivent être capables de s'adapter à des situations changeantes évaluées à partir d'algorithmes. Au-delà de leur utilisation industrielle, on les retrouve de plus en plus dans notre quotidien (nos machines domestiques, nos loisirs, nos véhicules, etc...).



Métier à tisser Jacquard- 1793



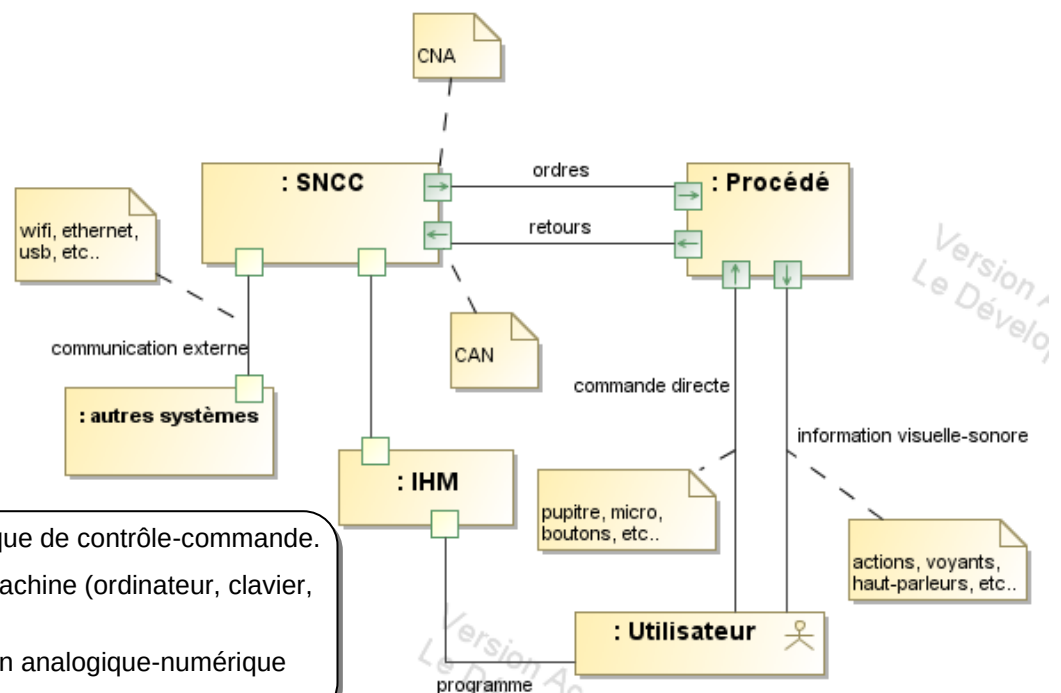
Robot NAO - 2008



Drone étanche SWELLPRO - 2021

1.2 Structure générale des SNCC :

La structure des SNCC peut varier suivant les applications. On peut néanmoins proposer la représentation générale sous la forme du diagramme de Bloc interne (ibd) ci-dessous :



SNCC : Système numérique de contrôle-commande.

IHM : Interface homme-machine (ordinateur, clavier, console spécifique, etc..).

CAN et **CNA** : Conversion analogique-numérique ou numérique-analogique.

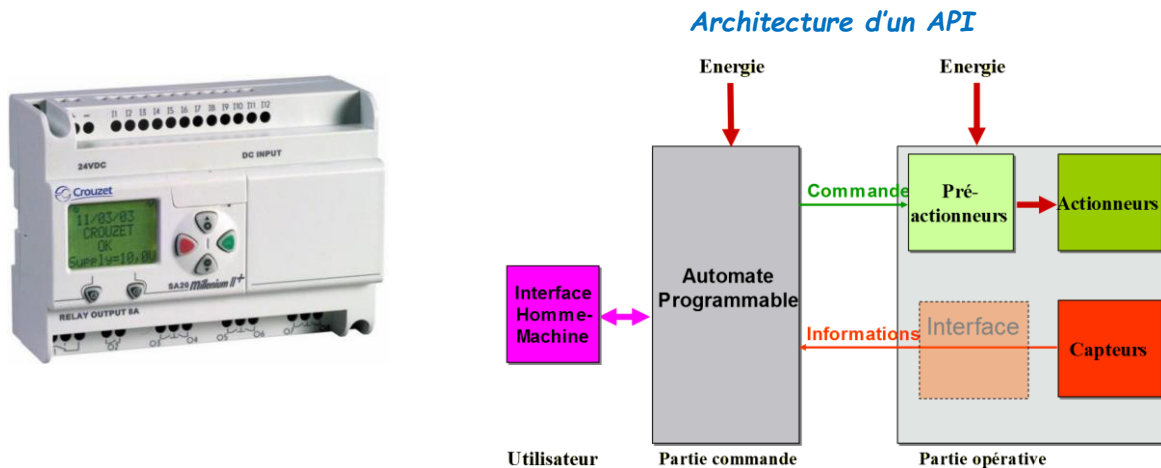
1.3 Solutions technologique pour les SNCC :

Distinguons quatre grandes familles :

- **Les Automates programmables industriels (API) :**

Ce sont des dispositifs programmables, destinés principalement à la commande de processus industriels. A partir d'informations électrique d'entrée provenant de **capteurs** ou de consignes humaine (via une **IHM**), l'automate enverra, sur la base d'un programme informatique, des ordres électriques vers ses sorties. Les entrées et sorties de l'automate sont de type **analogique** à courant faible. Pour pouvoir piloter les **actionneurs** il faudra en général convertir les énergies de commande de sorties en énergie de puissance, via des **préactionneurs** (**démo en TP**).

Leur structure se compose principalement d'un micro-processeur, de mémoires et d'interfaces d'**entrée-sortie**.

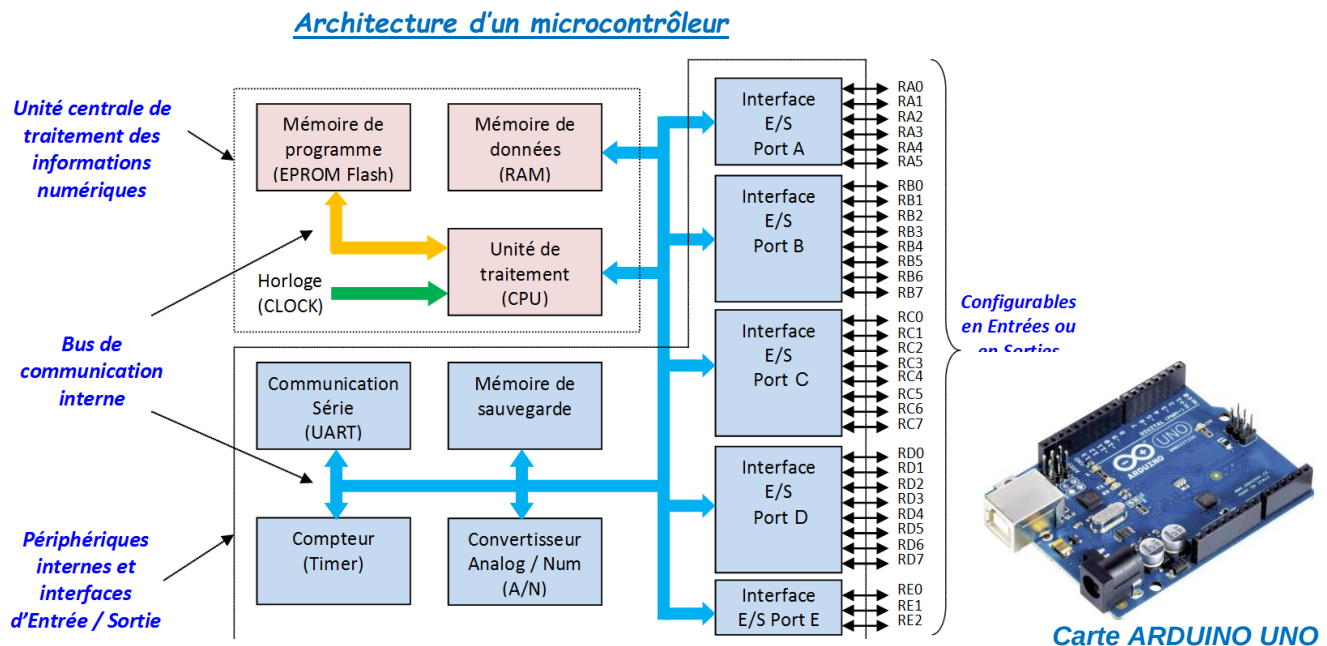


- **Les Microcontrôleur :**

Ils sont dédiés au pilotage de systèmes automatiques (robot, électroménager, drone, téléviseur, etc...).

La structure interne d'un microcontrôleur est configurée de façon définitive, elle n'est donc pas évolutive. Les performances sont très variables suivant les modèles et les applications. Cependant, les microcontrôleurs répondent à une architecture globale composée d'un microprocesseur, de mémoire non volatile (conservée hors alimentation : **ROM**), dans laquelle le programme (modifiable) sera implanté, de mémoire volatile (**RAM**) servant au calcul durant l'exécution, de bus de données, et de ports de communication entrée-sortie.

De petite taille et légers, ils conviennent bien aux systèmes embarqués.



- **Les ordinateurs (voir cours d'informatique) :**

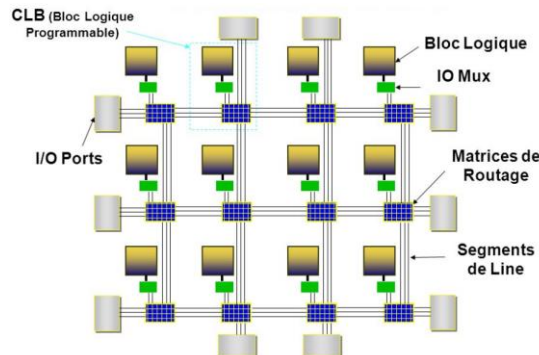
Un ordinateur permet de faire fonctionner un **OS** (operating system) et par conséquent de réaliser des tâches très variées en s'appuyant notamment sur des logiciels.

Ils adoptent la même base qu'un microcontrôleur (microprocesseur, **ROM**, **RAM**, bus de données, ports entrée-sortie) étoffée de multiples éléments complémentaires (processeurs dédiés, carte vidéo, carte son, etc...).

- **FPGA (Field Programmable Gate Arrays) :**

Ils se composent d'un réseau matriciel de cellules logiques entièrement reconfigurables. Les connexions entre chaque cellule logique de bas niveau (réalisant des opérations logiques élémentaires) sont ainsi reprogrammables à volonté au travers de matrices de routage. Le très grand nombre (plusieurs centaines de milliers) de cellules et de connexion permet une multitude de combinaisons évolutives en fonction des besoins. Cette reconfiguration interne dynamique, leur confère une très grande flexibilité et leur compacité une faible consommation d'énergie.

Architecture d'un FPGA



2. SYSTEMES DE NUMERATION (vus en cours d'informatique) :

Introduction : Si les SNCC sont dédiés le plus souvent à la gestion de systèmes analogiques, leur fonctionnement interne est quant à lui numérique. C'est-à-dire qu'ils manipulent des nombres. Au niveau élémentaire, cela signifie que le courant passe : **1** ou ne passe pas **0**. Ainsi, la combinaison de contacts électriques dans un SNCC se traduira en une multitude de 1 et de 0 appelés **bit** (binary digit). Afin de faciliter la manipulation du très grand nombre de bits, on a tendance, tant en informatique, qu'en électronique numérique à coder l'information en **base 16**, appelée aussi **base hexadécimale** et correspondant à des paquets de quatre bits.

Notons que l'unité de mesure du nombre de bits est représentée par des paquets de 8 bits, appelés **octets**.

Le traitement des SNCC nécessite donc quelques connaissances élémentaires sur le fonctionnement des systèmes de numération binaire et hexadécimal et leurs conversions. Même si ces notions ont été abordées en cours d'informatique, il est bon de rappeler les principaux mécanismes...

2.1 Système binaire :

Principe : Ce système, composé de 0 et de 1 répond à la numération illustrée ci-contre sur trois bits.

On appellera **LSB** (Least Significant Bit) ou **bit de poids faible**, le bit le plus à droite ;

On appellera **MSB** (Most Significant Bit) ou **bit de poids fort**, le bit le plus à gauche.

Changement de base :

• **Binaire → décimal :**
$$n_p n_{p-1} \dots n_1 n_0_2 = \left(\sum_{i=0}^p n_i \times 2^i \right)_{10}$$
 où i est appelé poids du bit n_i .

Décimal	Binaire pur
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Exemples :

$$(1001110)_2 = (2^1 + 2^2 + 2^3 + 2^6) = 78$$

• **Décimal → binaire :** Nous venons de voir qu'un nombre pouvait s'écrire sous la forme

$$\begin{aligned} N_{10} &= \sum_{i=0}^p n_i \times 2^i = n_p \times 2^p + n_{p-1} \times 2^{p-1} + \dots + n_1 \times 2^1 + n_0 \times 2^0 \\ &= 2 n_p \times 2^{p-1} + n_{p-1} \times 2^{p-2} + \dots + n_1 + n_0 \end{aligned}$$

$$N_{10} = 2 \times (\text{quotient}) + \text{reste}$$

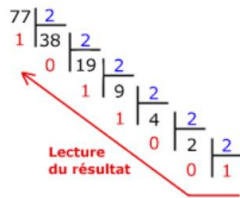
On notera un nombre N de base b : $(N)_b$, ex : $(11)_2 = (3)_{10}$.

Ce qui correspond à la division euclidienne de N_{10} par 2, avec un reste égal à 1 ou 0 !

Pour obtenir la conversion d'un nombre N exprimé en base 10 vers la base 2, il suffira alors de réaliser p divisions

successives de $N_{10} = \sum_{i=0}^p n_i \times 2^i$, le reste donnant à chaque division j le bits de poids $i=p-j$.

Exemple :



$$(1001101)_2 = (2^0 + 2^2 + 2^3 + 2^6) = (77)_{10}$$

Addition binaire :

L'addition binaire reprend bien entendu la logique de l'addition décimale, soit :

$$\begin{array}{r} 100010 \\ + 11011 \\ \hline 111101 \end{array} = \begin{array}{r} 34 \\ + 27 \\ \hline 61 \end{array}$$

2.2 Système Hexadécimal :

Principe : Ce système est composé de chiffres de 0 à 9 et de lettres de A à F, soit 16 éléments, et répond à la numération ci-contre :

Changement de base :

- **Hexa → décimal :** On reprendra la méthode binaire → décimal.

Exemple :

$$(3B)_{16} = (11 \times 16^0 + 3 \times 16^1) = 59$$

- **Binaire → hexadécimal :**

Là réside l'intérêt de la base 16...

En remarquant que : $16 = 2^4$

On peut écrire que : $\log_2(16) = 4 \log_2(2)$

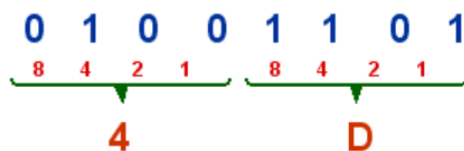
Cette égalité revient à dire qu'un chiffre dans la base 16, correspond exactement à quatre chiffres dans la base 2.

Il suffit ainsi de découper le mot binaire en « **quartets** », puis de convertir chaque « quartet » en base 16...

Notons par ailleurs qu'un **octet** (mot de 8 bits) correspond exactement à deux chiffres hexadécimaux.

Décimal	Hexadécimal
0	00
1	01
2	02
3	03
4	04
5	05
6	06
7	07
8	08
9	09
10	0A
11	0B
12	0C
13	0D
14	0E
15	0F
16	10
17	11

Exemple :



$$(4D)_{16} = (13 \times 16^0 + 4 \times 16^1) = (77)_{10}$$

3. QUELQUES CODES ELEMENTAIRES :

Introduction : Si tous les systèmes de numération correspondent à la même logique de comptage numérique, chaque code répond à des règles particulières. Ces dernières pourront être traduites par de simples tables de vérité comme par des algorithmes complexes.

Découverte dans la ville de Rosette durant la campagne d'Egypte de Napoléon en 1799, cette pierre qui comporte un texte écrit en deux langues : Hiéroglyphes et Grec ancien va permettre le décryptage de l'égyptien ancien.



Pierre de Rosette,
IIe siècle av.J.C

3.1 Codes binaires positifs :

- **Code binaire réfléchi ou code Gray :**

Codage sur 3 bits...

Principe : Ce code utilise la base 2, mais à la différence du binaire pur, seul un bit change d'état lorsque l'on évolue d'une unité (**illustration ci-contre**). On retrouvera cette propriété dans le fonctionnement logique des codeurs incrémentaux optiques (**voir TD ?**). Par ailleurs, ce code présente comme indiqué dans son nom, une symétrie dans sa construction.

Décimal	Binaire pur	Binaire réfléchi
0	000	0 0 0
1	001	0 0 1
2	010	0 1 1
3	011	0 1 0
4	100	1 1 0
5	101	1 1 1
6	110	1 0 1
7	111	1 0 0

Ingénieur Américain, il mettra au point le code portant son nom.



Frank GRAY, 1887-1969

- **Code thermomètre :**

Principe : Utilisé notamment dans les **convertisseurs analogique-numériques unipolaires (CAN) de type Flash (voir 4.3)**, ce code s'écrit avec 2^p-1 bits ; où p est le nombre de bits du plus grand nombre binaire à coder. Le principe décrit ci-contre, consiste à « allumer » pour un nombre $(N)_{10}$, N bits en partant du **LSB**.

3.2 Codage des nombres binaires relatifs :

- **Code binaire signé :**

Principe : Le code binaire signé est obtenu en rajoutant un bit de signe devant le **MSB** au code binaire naturel. Pour un bit égale à 0, le nombre est positif et il est négatif pour un bit égale à un. Le zéro peut être alors positif ou négatif, et présente ainsi deux codages.

Le code binaire signé est peu propice aux opérations arithmétiques.

- **Code binaire décalé :**

Code fréquemment utilisé dans les CAN...

Principe : Le code binaire décalé consiste, comme son nom l'indique, à décaler le code binaire naturel. Il permet de compter de -2^{N-1} à $2^{N-1}-1$. Le **MSB** fait là aussi office de bit de signe, mais dans une signification opposée au binaire signé.

- **Code complément à 2 :**

Principe : Le code complément à 2 correspond au code binaire décalé avec complémentarité du bit de signe. Pour les nombres positifs on retrouve le code binaire naturel.

Ce code doit son succès à sa parfaite adaptation aux opérations arithmétiques, de par sa double propriété : **les nombres positifs** (évalué par un zéro au MSB) **sont codés par le binaire naturel** et **les nombres strictement négatifs N_{10} sont codés par le nombre $N_{10}+2^n$** , où n est le nombre de bits du codage (**voir TD ?**).

3.3 Quelques autres codes

Il existe tant de codes qu'il serait donc vain de vouloir tous les présenter. On peut néanmoins citer en plus des codes binaires précédents quelques grandes familles, dont certains seront abordés en TD.

- **Code ASCII (American Standard Code For Information Interchange) :**

Apparu dans les années 60, il représente la norme de codage la plus influente des $2^7 = 128$ (codé sur 7 bits) caractères les plus courants pour l'échange d'information en informatique. On y retrouve notamment les chiffres arabes, l'alphabet latin minuscule et majuscule ainsi que divers symboles mathématiques et de ponctuation.

- **Code 2 parmi 5 (voir TD1) :**

Utilisé par la poste (bâtonnets rouges), il fait partie de la famille des codes à barres. Il se compose de groupes de cinq barres (toujours 2 épaisses et 3 fines parmi les 5) délimitées par trois barres représentant un « START » et un « STOP » de chaque côté. Leurs combinaisons permettent de coder les $C_5^2 = 10$ chiffres.

- **Le code EAN ou GENCODE (voir TD2) :**

A la différence du code 2 parmi 5, non seulement les barres, mais aussi les espaces codent l'information. Il possède par conséquent une plus grande densité d'information. Il est notamment utilisé dans le commerce et la distribution.

- **Code 2D QR (pour Quick Response) :**

Dernier né de la famille des codes à barres, il fut créé par un ingénieur japonais en 1999 afin de faciliter la gestion des pièces détachées dans les usines Toyota. Publié sous format libre, il sera concurrencé par des versions payantes comme le **Flashcode** ou le **2D-Doc**. Son codage se fait en deux dimensions offrant ainsi une densité d'information encore plus importante que le code EAN. Par ailleurs, les codes correcteurs (algorithmes Reed-Solomon) qui lui sont associés lui garantissent une très grande fiabilité de lecture. On le retrouve aujourd'hui dans bon nombre d'applications de smartphone...

Décimal	Binaire pur	Code thermomètre
0	000	0000000
1	001	0000001
2	010	0000011
3	011	0000111
4	100	0001111
5	101	0011111
6	110	0111111
7	111	1111111

Décimal	Binaire signé	Binaire décalé	Complément à 2
3	011	111	011
2	010	110	010
1	001	101	001
0	000/100	100	000
-1	101	011	111
-2	110	010	110
-3	111	001	101
-4	-	000	100

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	CTRL-G	32	20	Space	64	40	@	96	60		
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of transmit	EOT	CTRL-D	36	24	\$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	ACKnowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	3A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	3B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	3C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	3D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	3E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	3F	/	79	4F	O	111	6F	o
16	10	Data line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of transmit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	FS	CTRL-\	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	GS	CTRL-]	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	RS	CTRL-^	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	US	CTRL-~	63	3F	?	95	5F	~	127	7F	DEL

Code ASCII



Code 2 parmi 5



GENCODE



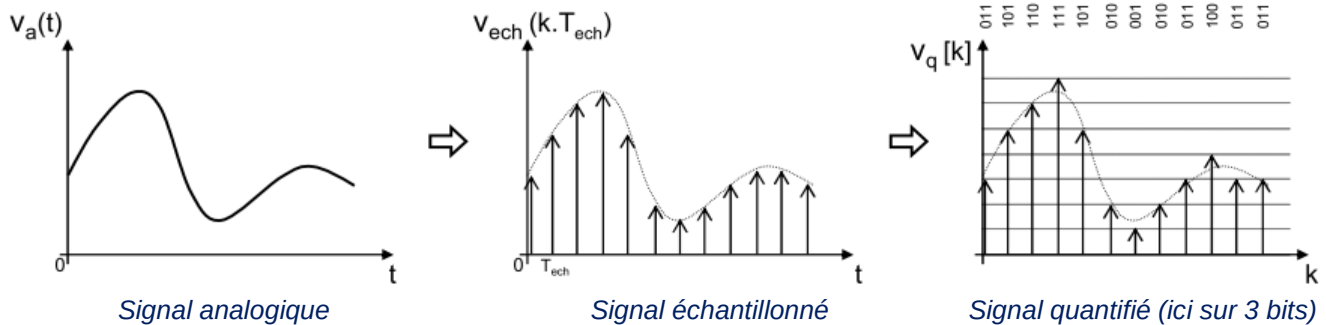
QR Code

4. NOTIONS DE CONVERSION ANALOGIQUE-NUMERIQUE :

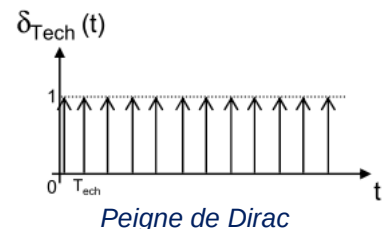
Introduction : Comme nous l'avons vu précédemment, nos **SNCC** sont de type numérique, ils ne manipulent que des nombres en interne. Pourtant, leur fonction est de commander des systèmes physiques qui eux sont **analogique**. Sans rentrer dans le détail des conversions numérique-analogique **CNA** ou analogique-numérique **CAN**, nous présenterons ici les concepts de base de ces conversions ...

4.1 Principe :

On cherche à associer une valeur **numérique, donc discrète**, à une grandeur **analogique, donc continue**, et inversement. Conceptuellement, on peut distinguer 3 étapes : **l'échantillonnage**, **la quantification** et **le codage**.

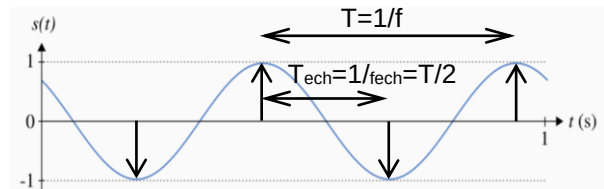


L'obtention d'un signal échantillonné $x_{ech}(k.T_{ech})$ à partir d'un signal analogique $x(t)$ peut être modélisé mathématiquement dans le domaine temporel par la multiplication de $x(t)$ à un **peigne de Dirac** de période T_{ech} , noté : $\delta_{T_{ech}}(t)$, illustré ci-contre et pouvant s'écrire : $x_{ech}(k.T_{ech}) = x(t) \cdot \delta(t) = x(t) \sum_k \delta(t - k.T_{ech})$



Sans rentrer là non plus dans le détail de l'analyse spectrale, on peut aisément comprendre que pour saisir suffisamment correctement le signal analogique, la fréquence doit être suffisante...

Pour illustrer cette notion, considérons un signal analogique sinusoïdal $s(t)$ de fréquence f , illustré ci-contre. On comprend immédiatement que la fréquence d'échantillonnage minimale : f_{ech} qui permettra de retranscrire ce message est le double de f .



Cette propriété s'énonce sous la forme du **théorème de Shannon** : $f_{ech} > 2f_{max}$, où f_{max} est la fréquence maximale de l'événement analogique que l'on cherche à numériser.

A titre d'exemple, la plage des fréquences audio que nous percevons va de 20 Hz à 20kHz, ce qui explique le choix de la fréquence d'échantillonnage des CD qui a été fixée à 44,1 kHz (légère marge).

4.2 Quantification :

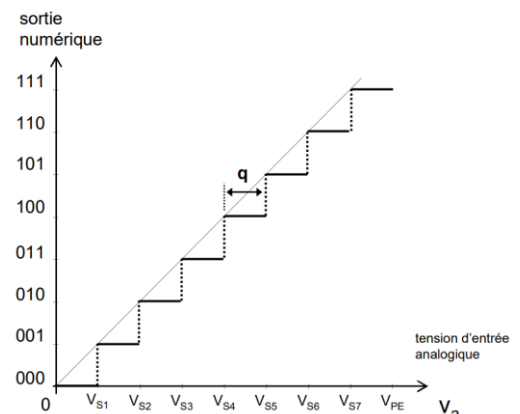
Le pas de quantification, et donc la précision d'un CAN dépendent du nombre de bits en sortie, appelé **résolution**. Pour un CAN à n bits, le nombre d'états possibles en sortie est 2^n , ce qui permet d'exprimer des signaux numériques de 0 à 2^n-1 en code binaire naturel.

Un CAN est également caractérisé par la plage de variation de la tension analogique d'entrée, appelée **Pleine Echelle** (FS pour Full Scale en anglais) et que nous noterons V_{PE} . La pleine échelle est divisée en autant de plages d'égale dimension (cas de la quantification uniforme) qu'il y a d'états possibles de la sortie numérique. Chaque plage est ainsi associée à un code numérique représentant la tension analogique d'entrée. La figure ci-contre représente la caractéristique de transfert idéale (sans défaut) en escalier d'un CAN à 3 bits.

On définit le **quantum q**, correspondant au **LSB** comme étant la dimension de ces plages.

On l'obtient par :

$$q = \frac{V_{PE}}{2^n}$$

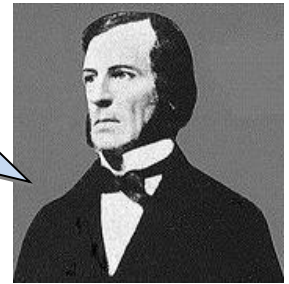


5. BASES DE L'ALGÈBRE LOGIQUE :

Introduction : L'**algèbre logique combinatoire** aussi appelée **algèbre de Boole** fut initiée en 1854 par le mathématicien britannique **George Boole** (illustration ci-contre).

Elle permet de modéliser des raisonnements logiques, en exprimant un « état » vrai ou faux représenté respectivement par un 1 ou un 0.

Mathématicien et philosophe Britannique né à Lincoln, il est le créateur de la logique algébrique moderne qui porte aujourd'hui son nom.

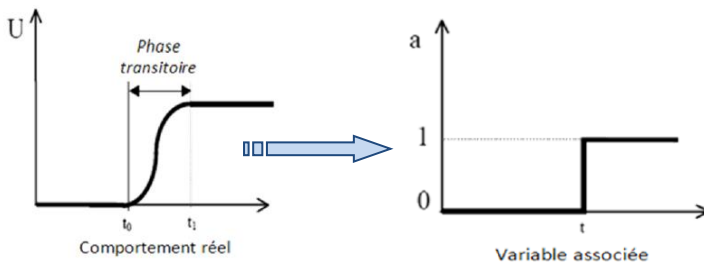


George BOOLE 1815-1864

5.1 Principe :

L'algèbre logique combinatoire s'appuie sur trois axiomes composés de deux lois : la conjonction (fonction **ET**) et la disjonction (**OU**) et une opération appelée complémentation (fonction **NON**).

- **Etat logique** : L'état logique est représenté soit par un **1 (vrai)**, soit par un **0 (faux)**.
- **Variable logique** : C'est une grandeur, représentée par un symbole (lettre ou signe) qui peut prendre les seules valeurs 0 ou 1 suivant son état logique, et en fonction de certaines conditions.
- **Fonction logique** : C'est une combinaison de variables reliées par des opérateurs logiques. Une fonction logique, tout comme les variables logiques, ne peut prendre que deux valeurs 0 ou 1.
- **Interprétation analogique** : Les valeurs 0 ou 1 d'une variable ou d'une fonction logique correspondent d'un point de vue technologique au seuil d'une grandeur analogique associée (ex : le courant passe : état 1, le courant ne passe pas : état 0) sans tenir compte du régime transitoire.



Niveau logique	Exemples de seuils analogiques		
	électrique	pneumatique	hydraulique
0	0V	0 bars	0 bars
1	+5V	+4 bars	+100 bars

5.2 Opérateurs logiques élémentaires :

- **La conjonction ou Fonction ET**, appelée aussi INTERSECTION ou PRODUIT LOGIQUE : Pour deux variables booléennes a et b, la valeur de **a ET b**, notée **a.b** sera 1 si a et b possèdent toutes les deux la valeur 1.
- **La disjonction ou Fonction OU**, appelée aussi UNION ou SOMME LOGIQUE : Pour deux variables booléennes a et b, la valeur de **a OU b**, notée **a + b** sera 1 si a ou b, ou les deux possède(nt) la valeur 1.
- **La complémentation ou opération NON** : Pour une variable booléenne a, la valeur de **NON (a)**, notée **$\bar{a}$** sera 1 si la valeur de a est 0 et 0 si la valeur de a est 1.

5.3 Propriétés de base de l'algèbre de Boole :

Propriétés	Application à la fonction OU	Application à la fonction ET
Commutativité	$a + b = b + a$	$a.b = b.a$
Associativité	$(a + b) + c = a + (b + c) = a + b + c$	$(a.b).c = a.(b.c) = a.b.c$
Double distributivité	$a + (b.c) = (a + b).(a + c)$	$a.(b + c) = (a.b) + (a.c)$
Idempotence	$a + a + \dots + a = a$	$a.a. \dots .a = a$
Eléments neutres	$a + 0 = a$	$a.1 = a$
Complément	$a + \bar{a} = 1$	$a.\bar{a} = 0$
Eléments absorbants	$a + 1 = 1$	$a.0 = 0$

5.4 Principe de dualité :

Toute expression logique demeure vraie si l'on remplace OU par ET et 0 par 1 et réciproquement.

Exemples ci-contre :

$$\begin{aligned}
 a + b &= b + a \Leftrightarrow a.b = b.a \\
 a + \bar{a}.b &= a + b \Leftrightarrow a.(\bar{a} + b) = a.b \\
 a + 1 &= 1 \Leftrightarrow a.0 = 0
 \end{aligned}$$

5.5 Théorème de DE MORGAN :

$$\overline{a+b} = \bar{a}\bar{b} \qquad \overline{a.b} = \bar{a} + \bar{b}$$

Ce théorème peut se généraliser à n variables booléennes en complémentant dans une expression quelconque les variables et en permutant les opérateurs ET et OU.

Mathématicien
Britannique né à Madura
(Inde) co-fondateur avec
BOOLE de la logique des
classes et des relations.



Augustus DE MORGAN
(1806-1871)

5.6 Autres opérateurs logiques essentiels :

Si toutes les expressions logiques peuvent s'écrire à l'aide des opérateurs logiques de base, il est intéressant de définir d'autres opérateurs plus globaux et fonction de ces premiers ; Voyons les principaux...

a	b	f ₀	f ₁	f ₂	f ₃	f ₄	f ₅	f ₆	f ₇	f ₈	f ₉	f ₁₀	f ₁₁	f ₁₂	f ₁₃	f ₁₄	f ₁₅
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Outre les fonctions constantes : f_0 et f_{15} (ne dépendant ni de a ni de b) et égalité : $f_3 = a$ et $f_5 = b$, notons :

- Deux fonctions complément : $f_{12} = \bar{a}$ et $f_{10} = \bar{b}$.
- Une fonction **ET** : $f_1 = a.b$.
- Une fonction **OU** : $f_7 = a + b$.
- Une fonction NON ET ou **NAND** : $f_{14} = a/b = \overline{a.b}$
- Une fonction OU exclusif ou **XOR** : $f_6 = a.\bar{b} + \bar{a}.b$; $f_6 = 1$ si $a = 1$ ou $b = 1$ mais pas lorsque $a = b = 1$.
- Une fonction NON OU, **NOR** ou NI : $f_8 = \overline{a+b} = \bar{a}.\bar{b}$ c'est à dire que $f_8 = 1$ si ni $a = 1$, ni $b = 1$.

Les fonctions NAND et NOR sont des
fonctions universelles (**voir TD1**).

5.7 Simplification de fonctions :

Il est souvent possible d'écrire une fonction logique sous une expression équivalente plus simple. Pour cela, deux méthodes peuvent être utilisées en fonction de la complexité de l'écriture initiale :

- L'expression logique est simple (composée au plus de 3 variables) : Les simplifications pourront en général être rapidement trouvée par l'algèbre logique, dont on précise quelques reflexes ci-dessous ;
- L'expression logique est plus complexe : On utilise le tableau de Karnaugh (**hors programme**).

Voici donc quelques reflexes à noter pour la simplification de fonctions logiques simples :

→ Ecrire les produits dans l'ordre alphabétique afin de les comparer plus facilement ;

→ Faire apparaître des éléments neutres ou absorbant afin de les annuler ; **Exemple ci-contre** :

$$a.b + \bar{a}.b = b(a + \bar{a}) = b.1 = b$$

$$a + a.b = a(1 + b) = a$$

→ Rajouter des termes déjà existants (idempotence) afin d'obtenir des regroupements supplémentaires ;

Exemple ci-contre :

$$a.b.c + \bar{a}.b.c + a.\bar{b}.c = \underbrace{a.b.c + \bar{a}.b.c}_{b.c} + \underbrace{a.b.c + a.\bar{b}.c}_{a.c} = b.c + a.c$$

→ Supprimer un terme déjà inclus ; **Exemple ci-contre** :

$$a.b + \bar{b}.c + a.c = a.b + \bar{b}.c$$

Illustration d'une simplification d'expression booléenne avec un tableau de Karnaugh :

$$\underbrace{\bar{a}.\bar{b}.c + \bar{a}.b.c}_{\bar{a}.c} + \underbrace{\bar{a}.b.\bar{c} + a.b.\bar{c}}_{\bar{a}.b} + \underbrace{a.b.c + a.\bar{b}.c}_{a.c}$$

$$= \bar{a}.c + \bar{a}.b + a.c = \bar{a}.b + c$$

a 0 0 1 1
b 0 1 1 0

c				
0	0	1	0	0
1	1	1	1	1

Ingénieur Américain connu pour la
table portant son nom, développée
pour le laboratoire Bell en 1953...



Maurice KARNAUGH
(né en 1924)